LTMB13 - IDMC - CAI Modules for Understanding: D8 - 26.10.2026

Cloud Application Integration (CAI) Track Modules:

- CAI Architecture
- API management
- Process designer
- Service Connectors
- Fault handling
- Monitoring
- Asset management

Informatica IDMC – Cloud Application Integration (CAI)

Complete Guide: Concepts, Architecture, Business Use Cases, Practical Exercises & Interview Preparation

Track: Informatica Cloud Application Integration (CAI)

Audience: Data Engineers / Integration Developers

Level: Beginner to Intermediate

Understanding Approach: Concept → Business Scenario → Architecture → Implementation → Validation → Interview Questions

---

PART 1: Understanding Cloud Application Integration (CAI)

1. What is Cloud Application Integration?

Cloud Application Integration (CAI) is a service within Informatica Intelligent Data Management Cloud (IDMC) that enables organizations to connect applications, automate business processes, expose APIs, and orchestrate interactions between different systems.

In simple terms:

> CAI helps different business applications communicate with one another and automate activities without requiring users to perform every step manually.

Real-World Business Example: Online Shopping

Consider an e-commerce company.

A customer places an order on a shopping website.

Several business applications must communicate:

1. The shopping application receives the order.
2. The inventory application checks product availability.
3. The payment application processes the payment.
4. The warehouse application prepares the shipment.
5. The notification application sends confirmation to the customer.

Without integration, employees might need to update these systems manually.

With CAI, the complete workflow can be automated.

Business Architecture

```
                CUSTOMER
                   |
                   v
          E-COMMERCE WEBSITE
                   |
                   v
            REST API CALL
                   |
                   v
    INFORMATICA CLOUD APPLICATION
           INTEGRATION (CAI)
                   |
                   v
            PROCESS DESIGNER
                   |
```

```
          +----------+----------+
          |          |          |
          v          v          v
      INVENTORY   PAYMENT    CUSTOMER
       SYSTEM     GATEWAY      CRM
          |          |          |
          +----------+----------+
                     |
                     v
              ORDER CONFIRMED
                     |
                     v
            SEND NOTIFICATION
```

Why do companies use CAI?

Business Problem	CAI Solution
Applications operate independently	Connects applications through APIs and connectors
Manual approval processes	Automates approval workflows
Repeated manual data entry	Exchanges information between systems
Different applications use different interfaces	Integrates applications through services and APIs
Business transactions fail	Supports fault handling and recovery
Integration processes are difficult to monitor	Provides execution monitoring and logs
Applications need reusable services	Supports reusable process assets and service connectors

Interview Question 1: What is Informatica CAI?

Answer:

Informatica Cloud Application Integration is a cloud-based application integration service within IDMC. It is used to automate business workflows, connect applications, expose and consume APIs, and orchestrate business processes across cloud and on-premises systems.

It provides capabilities such as Process Designer, Service Connectors, API-based integration, fault handling, and process monitoring.

Key terminology: Application Integration, Orchestration, APIs, Business Process Automation.

PART 2: CAI vs CDI – Understanding the Difference

Understand the difference between **Cloud Data Integration (CDI) and Cloud Application Integration (CAI).**

Feature	Cloud Data Integration (CDI)	Cloud Application Integration (CAI)
Primary purpose	Data integration and transformation	Application integration and process orchestration
Processing style	Typically batch or data pipeline execution	Event-driven, synchronous, or asynchronous processing
Main development tool	Mapping Designer	Process Designer
Common input	Database tables, files, cloud storage	API requests, application events, service responses
Main operations	Filter, Join, Expression, Aggregator	Decision, Assignment, Service Call, Parallel Paths
Typical business use	Load customer records into a data warehouse	Process a customer order across multiple applications
Output	Transformed datasets	Business responses, service interactions, workflow outcomes
Execution	Mapping tasks and data integration runtime	Application integration process runtime
Example	Load 100,000 sales records into Snowflake	Validate and approve a customer order

Important distinction

CDI focuses primarily on moving and transforming data.

CAI focuses primarily on coordinating applications and automating business processes.

However, CAI can also invoke CDI assets, allowing business workflows to trigger data integration activities.

Business Example

A company wants to load daily sales records into Snowflake.

CDI is appropriate for performing the data movement and transformations.

A company wants to automatically start a sales data pipeline after receiving an API request and then notify another system about the outcome.

CAI can orchestrate this workflow.

Interview Question 2: What is the difference between CDI and CAI?

Answer:

CDI is primarily used to extract, transform, and load data between systems. CAI is primarily used to connect applications, consume and expose APIs, and automate business workflows.

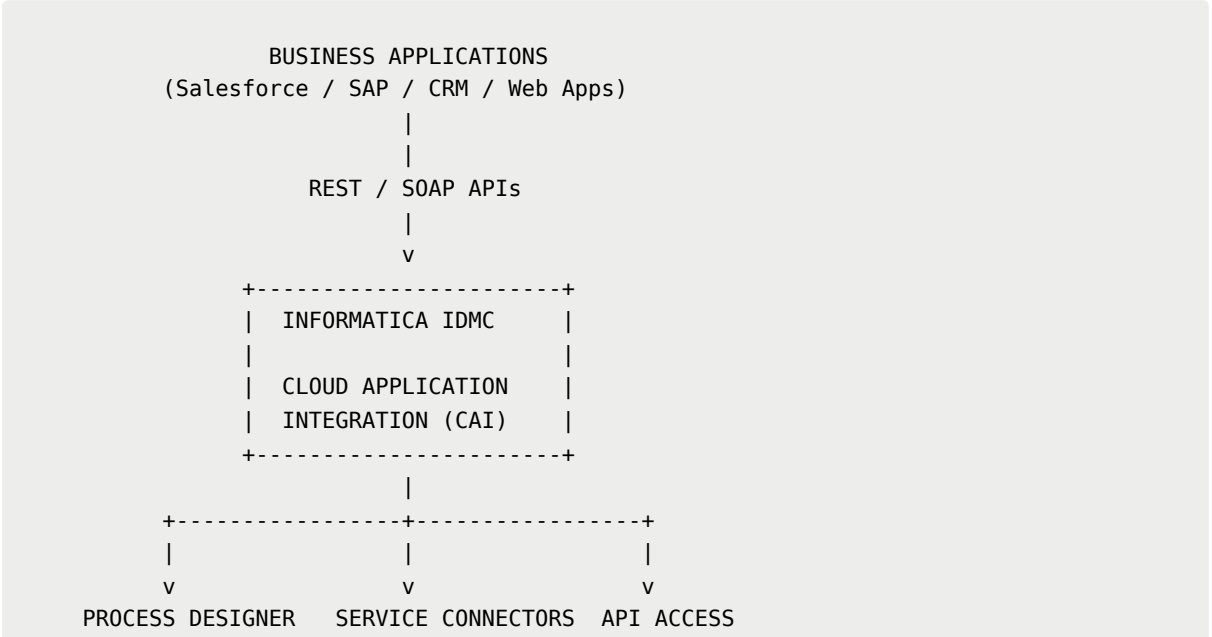
For example, CDI can load sales transactions into Snowflake, while CAI can receive an API request, trigger the sales integration process, and return or communicate the execution result.

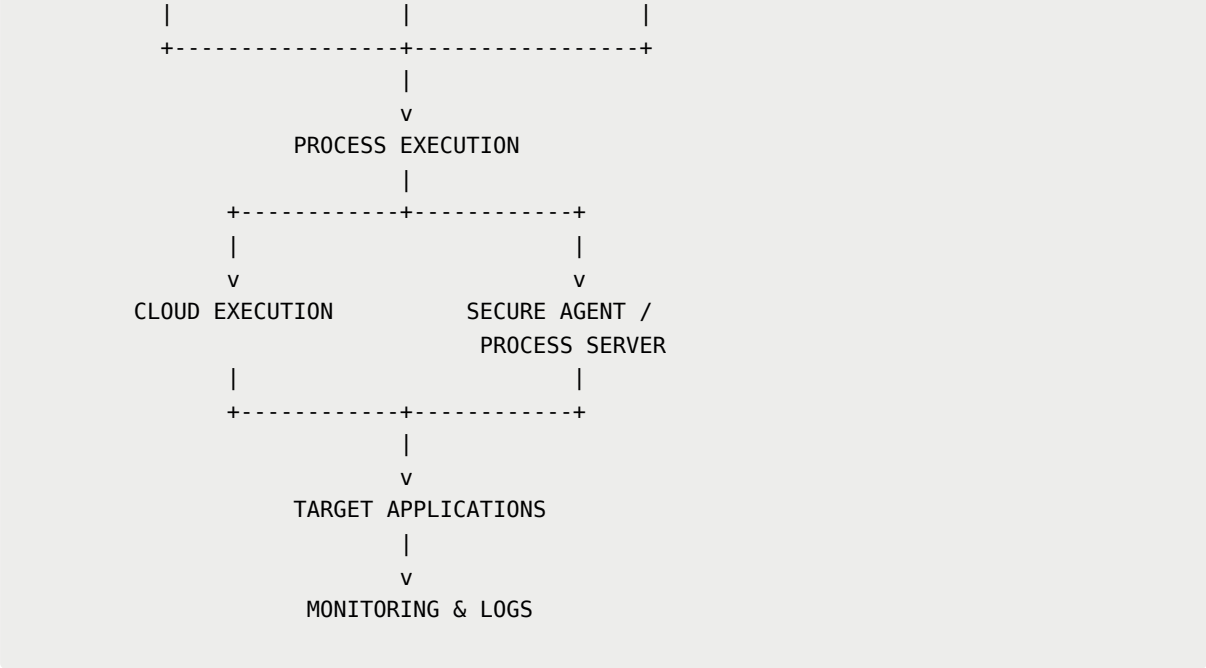
PART 3: CAI Architecture

3.1 Understanding CAI Architecture

CAI architecture consists of components that support process development, application connectivity, execution, API exposure, and monitoring.

Simplified CAI Architecture





This is a conceptual architecture. The actual runtime and available capabilities depend on the organization's CAI configuration, licensing, and deployed services.

3.2 Major Components of CAI Architecture

Component 1: Informatica IDMC

IDMC is the cloud platform that provides services for data management, integration, governance, and related capabilities.

CAI is one of the services available within the platform.

Purpose: Provides a centralized environment for designing, deploying, and managing application integration solutions.

Component 2: Process Designer

Process Designer is the visual development environment used to build application integration processes.

A developer can create a workflow using steps such as:

- Start
- Assignment
- Decision

- Service
- Parallel Paths
- Throw
- End

Example:

```
START
  |
  v
RECEIVE ORDER
  |
  v
CHECK ORDER AMOUNT
  |
  v
DECISION
  |
  +---- Amount > 10000 ----> MANAGER APPROVAL
  |
  +---- Amount <= 10000 ---> AUTO APPROVAL
  |
  v
RETURN RESULT
  |
  v
END
```

Component 3: Service Connectors

Service Connectors define how CAI interacts with external services and APIs.

They describe service operations, request structures, response structures, and related connectivity information.

For example, a service connector can be configured to invoke a REST API that returns customer details.

Component 4: Connections

Connections provide the configured access information needed to communicate with external systems.

Depending on the connector, configuration can include:

- Endpoint information
- Authentication details

- Credentials
- Connection properties
- Security settings

A connection is different from a service connector: the service connector describes the service interface, while the connection supplies the configuration needed to access it.

Component 5: Process Execution Runtime

The runtime executes deployed application integration processes.

Depending on the configuration, processes can execute using supported cloud execution services or a Secure Agent environment with the appropriate application integration runtime components.

Component 6: Monitoring

Monitoring helps developers and administrators investigate:

- Process execution status
- Successful and failed instances
- Fault information
- Execution details
- Process activity

Interview Question 3: Explain CAI architecture.

Answer:

CAI architecture includes the design environment, connectivity components, execution runtime, and monitoring capabilities.

Developers use Process Designer to create integration workflows and Service Connectors to communicate with external APIs. After deployment, supported runtime services execute the processes. Monitoring capabilities help developers track executions, analyze failures, and troubleshoot integration issues.

PART 4: API Management in CAI

4.1 What is an API?

API stands for **Application Programming Interface**.

An API allows one software application to communicate with another through a defined interface.

Example

A banking application needs customer account details.

Instead of directly accessing another application's database, it can call an API.

```
BANKING APPLICATION
        |
        v
GET CUSTOMER API
        |
        v
CUSTOMER MANAGEMENT SYSTEM
        |
        v
CUSTOMER DETAILS
```

Example request:

```
GET /customers/C101
```

Example response:

```
{
  "customer_id": "C101",
  "customer_name": "Ravi Kumar",
  "account_status": "ACTIVE"
}
```

The calling application does not need to know how the customer management system stores its data.

It only needs to understand the API contract.

4.2 REST API Fundamentals

REST stands for **Representational State Transfer**.

REST APIs commonly use HTTP methods to perform operations on resources.

HTTP Methods

HTTP Method	Purpose	Business Example
GET	Retrieve information	Get customer details
POST	Create or submit information	Create a new order
PUT	Replace a resource representation	Replace customer information
PATCH	Partially update information	Update customer phone number
DELETE	Delete a resource	Delete a temporary record

Example: Customer API

GET request

```
GET /api/customers/C101
```

Response

```
{
  "customer_id": "C101",
  "name": "Ravi Kumar",
  "city": "Chennai"
}
```

Common HTTP Status Codes

Status Code	Meaning	Example
200	OK	Customer details retrieved
201	Created	New customer created
400	Bad Request	Invalid input
401	Unauthorized	Missing or invalid authentication
403	Forbidden	Insufficient permissions
404	Not Found	Customer does not exist
500	Internal Server Error	Server-side processing failed

Status Code	Meaning	Example
503	Service Unavailable	Service temporarily unavailable

4.3 What is API Management?

API Management refers to managing the lifecycle, access, security, and usage of APIs.

It can include:

- Publishing APIs
- Securing APIs
- Controlling access
- Applying usage policies
- Monitoring API traffic
- Managing API versions

Important CAI distinction

Cloud Application Integration provides process orchestration and API-based integration capabilities.

Informatica API Center / API management capabilities, where available and licensed, provide additional API lifecycle and governance functions.

These capabilities should not be treated as identical products.

Business Example: Payment API

An organization exposes an API:

```
POST /api/payments
```

Request:

```
{
  "customer_id": "C101",
  "amount": 5000,
```

```
  "payment_mode": "UPI"
}
```

Response:

```
{
  "transaction_id": "TXN1001",
  "payment_status": "SUCCESS"
}
```

CAI can implement the business process behind this API.

For example:

```
API REQUEST
    |
    v
VALIDATE INPUT
    |
    v
CHECK PAYMENT MODE
    |
    v
CALL PAYMENT SERVICE
    |
    v
PROCESS RESPONSE
    |
    v
RETURN PAYMENT STATUS
```

Interview Question 4: What is the role of APIs in CAI?

Answer:

APIs enable CAI processes to communicate with external applications and expose integration functionality to other systems.

CAI can consume APIs through configured service interactions and can expose supported processes as callable services. This helps organizations automate workflows without tightly coupling applications.

PART 5: Process Designer

5.1 What is Process Designer?

Process Designer is a visual development environment used to create and configure business processes in Informatica CAI.

A process consists of steps that execute according to the business logic.

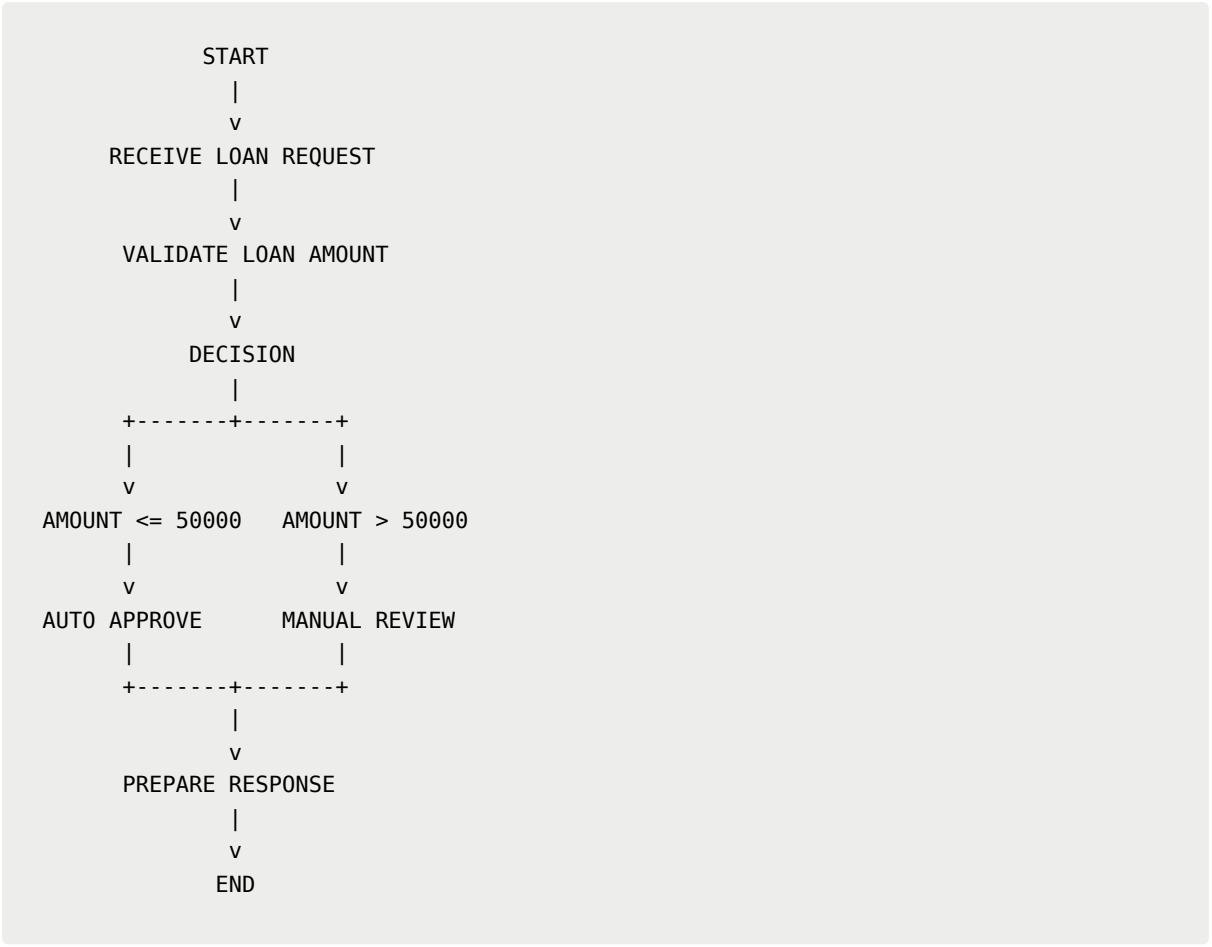
Example: Loan Approval

A bank receives loan applications.

Business rules:

- Loan amount up to ₹50,000: automatic approval.
- Loan amount above ₹50,000: manual review required.

Process Flow



5.2 Important Process Designer Steps

Process Step	Purpose	Example
Start	Defines process entry point	Receive loan request
Assignment	Assigns or calculates values	Calculate approval status
Decision	Evaluates conditions	Amount greater than ₹50,000?
Service	Invokes a configured service operation	Call customer API
Parallel Paths	Executes independent branches concurrently	Notify CRM and audit service
Throw	Raises a fault	Invalid loan amount
End	Completes the process	Return approval response

Additional process constructs and configuration options may be available depending on the CAI environment.

5.3 Process Input and Output

A process can accept input values and produce output values.

Example Input

```
{
  "loan_id": "L1001",
  "customer_id": "C101",
  "loan_amount": 25000
}
```

Business Rule

```
IF loan_amount <= 50000
    approval_status = "APPROVED"
ELSE
    approval_status = "MANUAL_REVIEW"
```

Example Output

```
{
  "loan_id": "L1001",
```

```
  "approval_status": "APPROVED"
}
```

Interview Question 5: What is the purpose of Process Designer?

Answer:

Process Designer is used to visually create and configure application integration workflows.

It allows developers to define process inputs, outputs, business rules, decision conditions, service calls, and exception-handling behavior.

It simplifies the development of business processes involving multiple applications.

PART 6: Practical Exercise 1 – Loan Approval Process

Objective

Create a simple CAI process that accepts a loan amount and returns an approval status.

Estimated duration: 45–60 minutes.

Prerequisites

- Access to an Informatica IDMC organization.
- Cloud Application Integration entitlement.
- Permission to create and publish processes.
- Access to an appropriate process execution runtime.

Step 1: Open CAI

1. Sign in to Informatica IDMC.
2. Open the Cloud Application Integration service.
3. Navigate to the design environment.

4. Select the required project or folder.
5. Create a new Process asset.

The exact menu labels can vary by IDMC release.

Step 2: Create the Process

Suggested process name:

`PRC_LOAN_APPROVAL`

Define the required process inputs and outputs.

Input fields:

Field	Type
loan_id	String
customer_id	String
loan_amount	Decimal

Output fields:

Field	Type
loan_id	String
approval_status	String

Step 3: Configure a Decision

Condition:

```
loan_amount <= 50000
```

True branch:

```
approval_status = "APPROVED"
```

False branch:

```
approval_status = "MANUAL_REVIEW"
```

Use the appropriate CAI expression syntax when configuring the actual process.

Step 4: Assign the Output

Populate the process output fields.

Step 5: Publish the Process

Validate and publish the process using the available runtime configuration.

Step 6: Test the Process

Test Case 1

Input:

```
{
  "loan_id": "L1001",
  "customer_id": "C101",
  "loan_amount": 25000
}
```

Expected status:

```
APPROVED
```

Test Case 2

Input:

```
{
  "loan_id": "L1002",
  "customer_id": "C102",
  "loan_amount": 100000
}
```

Expected status:

```
MANUAL_REVIEW
```

Validation

Confirm that the process returns the correct approval status for both cases.

Training outcome: Students understand process inputs, outputs, assignments, decisions, publishing, and execution.

PART 7: Service Connectors

7.1 What is a Service Connector?

A Service Connector defines how an Informatica CAI process communicates with an external service.

It describes the service interface and operations that the process can invoke.

Business Example: Weather API

A logistics company needs weather information before scheduling deliveries.

Its delivery process calls a weather API.

```
DELIVERY APPLICATION
        |
        v
CAI PROCESS
        |
        v
SERVICE CONNECTOR
        |
        v
WEATHER REST API
        |
        v
WEATHER RESPONSE
        |
        v
DELIVERY DECISION
```

Example API Request

```
GET /weather?city=Chennai
```

Illustrative response:

```
{
  "city": "Chennai",
  "temperature": 32,
  "condition": "Rainy"
}
```

The process can evaluate the returned weather condition and determine whether delivery requires additional review.

7.2 Service Connector vs Connection

Service Connector	Connection
Describes the external service interface	Configures access to the external service
Defines operations and data structures	Provides endpoint and authentication configuration
Can support reusable service definitions	Supplies environment-specific connection details
Used when designing service interactions	Used when configuring connectivity
Example: Customer lookup API operation	Example: Customer API endpoint and credentials

Interview Question 6: What is a Service Connector?

Answer:

A Service Connector is a reusable CAI asset that defines the interface of an external service, including its operations and request and response structures.

It allows application integration processes to invoke external services through configured connections.

PART 8: Practical Exercise 2 – REST API Integration

Business Scenario: Customer Verification

A retail company wants to verify whether a customer is active before accepting an order.

Objective

Create a CAI process that receives a customer ID, calls a customer verification API, and returns the customer's status.

Architecture

```
CUSTOMER REQUEST
       |
       v
CAI PROCESS
       |
       v
SERVICE CONNECTOR
       |
       v
CUSTOMER VERIFICATION API
       |
       v
CUSTOMER STATUS RESPONSE
       |
       v
CAI DECISION
       |
       +---- ACTIVE ----> ACCEPT
       |
       +---- INACTIVE --> REJECT
```

Example Request

```
{
  "customer_id": "C101"
}
```

Illustrative API Response

```
{
  "customer_id": "C101",
  "customer_status": "ACTIVE"
}
```

Expected Process Output

```
{
  "customer_id": "C101",
  "order_eligibility": "ACCEPTED"
}
```

Implementation Outline

1. Create a service connector for the customer verification API.
2. Configure the service endpoint and authentication.
3. Define the customer lookup operation.
4. Create a process named `PRC_CUSTOMER_VERIFICATION`.
5. Configure the customer ID as an input.
6. Add a Service step to call the verification API.
7. Add a Decision step to check the returned status.
8. Assign the eligibility result.
9. Publish and test the process.

Note: This exercise requires a reachable customer API or a mock API endpoint. The examples above describe the expected contract; they are not a live endpoint.

PART 9: Fault Handling

9.1 What is Fault Handling?

Fault handling is the mechanism used to detect and respond to errors during process execution.

In real business applications, failures can occur because of:

- Invalid input data
- Incorrect authentication
- Unavailable APIs
- Network interruptions

- Service timeouts
- Business rule violations
- Unexpected response formats

A well-designed process should respond appropriately instead of failing without useful information.

Business Example: Payment Processing

A customer submits a payment.

The payment gateway is temporarily unavailable.

Without appropriate fault handling:

```
PAYMENT REQUEST
      |
      v
PAYMENT API
      |
      v
SERVICE FAILURE
      |
      v
PROCESS FAILS
```

With fault handling:

```
PAYMENT REQUEST
      |
      v
PAYMENT API
      |
      v
SERVICE FAILURE
      |
      v
FAULT HANDLING
      |
      +---- LOG ERROR
      |
      +---- CLASSIFY FAILURE
      |
      +---- APPLY RECOVERY POLICY
      |
      +---- RETURN ERROR RESPONSE
```

9.2 Types of Faults

Fault Type	Description	Example
Validation fault	Input does not satisfy business rules	Negative payment amount
Authentication fault	Service authentication fails	Invalid API token
Authorization fault	User or process lacks permission	Access denied
Connection fault	External service cannot be reached	Network failure
Timeout fault	Service takes too long to respond	Payment gateway timeout
Business fault	A business condition prevents processing	Insufficient balance
System fault	Unexpected technical failure	Internal service error

9.3 Fault Handling Strategies

Strategy 1: Input Validation

Validate information before invoking an external service.

Example:

```
IF payment_amount <= 0
    REJECT REQUEST
```

Strategy 2: Error Logging

Record meaningful error information.

Example:

```
{
  "transaction_id": "TXN1003",
  "error_code": "INVALID_AMOUNT",
  "error_message": "Payment amount must be greater than zer
o"
}
```

Strategy 3: Retry

Retry an operation when the failure is temporary and retrying is safe.

For example, a read-only customer lookup may be retried after a transient timeout.

Important: Payment creation and other non-idempotent operations should not be blindly retried. Use transaction identifiers, idempotency controls, or status verification to prevent duplicate transactions.

Strategy 4: Return a Meaningful Response

Instead of exposing internal technical details, return a clear error message.

Example:

```
{
  "transaction_id": "TXN1003",
  "status": "FAILED",
  "reason": "Invalid payment amount"
}
```

Interview Question 7: What is fault handling in CAI?

Answer:

Fault handling is the mechanism used to manage errors that occur during process execution.

It allows developers to identify failures, capture diagnostic information, execute appropriate recovery logic, and return meaningful error responses.

Fault handling improves the reliability and maintainability of application integration processes.

PART 10: Practical Exercise 3 – Payment Validation with Fault Handling

Business Scenario

A banking application receives payment transactions.

The business rules are:

1. Payment amount must be greater than zero.
2. Payment mode must be valid.
3. Invalid requests must return a meaningful error.
4. Valid requests can proceed to the next processing stage.

Input

```
{
  "transaction_id": "TXN1001",
  "amount": 5000,
  "payment_mode": "UPI"
}
```

Allowed Payment Modes

```
UPI
CARD
NEFT
CASH
```

Process Flow

```
          START
            |
            v
      RECEIVE PAYMENT
            |
            v
      VALIDATE AMOUNT
            |
         +----+----+
         |         |
      INVALID     VALID
         |         |
         v         v
     ERROR      CHECK MODE
                   |
              +----+----+
              |         |
           INVALID     VALID
              |         |
              v         v
            ERROR    ACCEPT
```

```
                |         |
                +----+----+
                     |
                     v
                RETURN RESULT
                     |
                     v
                    END
```

Expected Results

Test Case	Amount	Payment Mode	Expected Status
TC01	5000	UPI	ACCEPTED
TC02	-2500	CARD	REJECTED
TC03	0	CASH	REJECTED
TC04	7500	INVALID	REJECTED
TC05	12000	CARD	ACCEPTED

Implementation Guidance

Use Process Designer to create the required input fields, Decision steps, Assignment steps, and outputs.

For a more advanced exercise, use a Throw step to raise a fault for invalid input and configure the corresponding fault handling behavior.

Interview Question 8: How do you handle invalid input in a CAI process?

Answer:

I would validate the input fields before invoking downstream services.

For example, in a payment process, I would verify that the amount is positive and the payment mode is supported.

If validation fails, I would configure the process to return an appropriate business error or raise a fault, depending on the API contract.

I would also capture relevant diagnostic information for troubleshooting.

PART 11: Monitoring in CAI

11.1 What is Monitoring?

Monitoring allows developers and administrators to observe process execution and troubleshoot problems.

After publishing and executing a process, developers need to know:

- Did the process execute successfully?
- Did the process fail?
- Which step failed?
- What was the error?
- How long did the execution take?
- What information is available for troubleshooting?

Monitoring provides visibility into process execution.

Example

A company processes 1,000 customer requests.

The monitoring information might show:

Metric	Value
Total Requests	1000
Successful	950
Failed	50
Success Rate	95%

These are illustrative metrics, not actual CAI monitoring results.

11.2 Monitoring Workflow

```
PROCESS EXECUTION
        |
        v
EXECUTION DETAILS
        |
        v
MONITORING
        |
        +---- SUCCESS
        |
        +---- FAILURE
```

```
         |
         +---- FAULT DETAILS
         |
         +---- EXECUTION INFORMATION
         |
         v
TROUBLESHOOTING
```

What should a fresher check when a process fails?

1. Identify the failed process instance.
2. Review its execution status.
3. Locate the failing activity or service call.
4. Read the available fault details.
5. Verify the input and relevant connection configuration.
6. Determine whether the issue is related to data, authentication, connectivity, or process logic.
7. Report the findings to the responsible team.

Interview Question 9: How do you monitor a CAI process?

Answer:

I would use the monitoring capabilities available in Informatica Cloud Application Integration to review process instances and their execution status.

If a process fails, I would inspect the available fault details and execution information to identify the failing step and understand the cause.

As a fresher, I would document the findings and escalate issues that require production configuration changes or elevated permissions.

PART 12: Asset Management

12.1 What is Asset Management?

Asset management refers to organizing and maintaining integration development objects throughout their lifecycle.

In CAI, these objects can include:

- Processes
- Service Connectors
- Connections
- Related reusable integration assets

Why is Asset Management Important?

Imagine an organization has 200 CAI processes.

Without proper organization, developers may struggle to:

- Find the correct process.
- Identify which connector a process uses.
- Understand the purpose of an asset.
- Manage changes safely.
- Promote tested assets between environments.
- Avoid unnecessary duplication.

Asset management helps maintain consistency and traceability.

12.2 Example Folder Structure

```
IDMC PROJECTS
      |
      +---- BANKING_INTEGRATION
      |          |
      |          +---- PROCESSES
      |          |
      |          +---- SERVICE_CONNECTORS
      |          |
      |          +---- CONNECTIONS
      |
      +---- RETAIL_INTEGRATION
                 |
                 +---- PROCESSES
                 |
                 +---- SERVICE_CONNECTORS
                 |
                 +---- CONNECTIONS
```

This is a suggested logical organization. Actual project and folder structures depend on how the organization manages IDMC assets.

Naming Standards

Asset Type	Suggested Name
Process	`PRC_PAYMENT_VALIDATION`
Process	`PRC_CUSTOMER_VERIFICATION`
Service Connector	`SC_CUSTOMER_API`
Service Connector	`SC_PAYMENT_API`
Connection	`CONN_CUSTOMER_API_DEV`
Connection	`CONN_PAYMENT_API_TEST`

Development Lifecycle

```
DEVELOPMENT
     |
     v
UNIT TESTING
     |
     v
TEST ENVIRONMENT
     |
     v
VALIDATION
     |
     v
PRODUCTION DEPLOYMENT
     |
     v
MONITORING
```

Production deployment should follow the organization's approval and change-management procedures.

Interview Question 10: Why is asset management important in CAI?

Answer:

Asset management helps organize, maintain, and reuse integration components such as processes, service connectors, and connections.

It supports consistent naming, easier maintenance, controlled changes, and smoother deployment between environments.

It also reduces the risk of developers creating duplicate or inconsistent integration assets.

PART 13: Complete End-to-End Business Case Study

Retail Order Processing Using CAI

This example combines the major CAI concepts into one business scenario.

Business Requirement

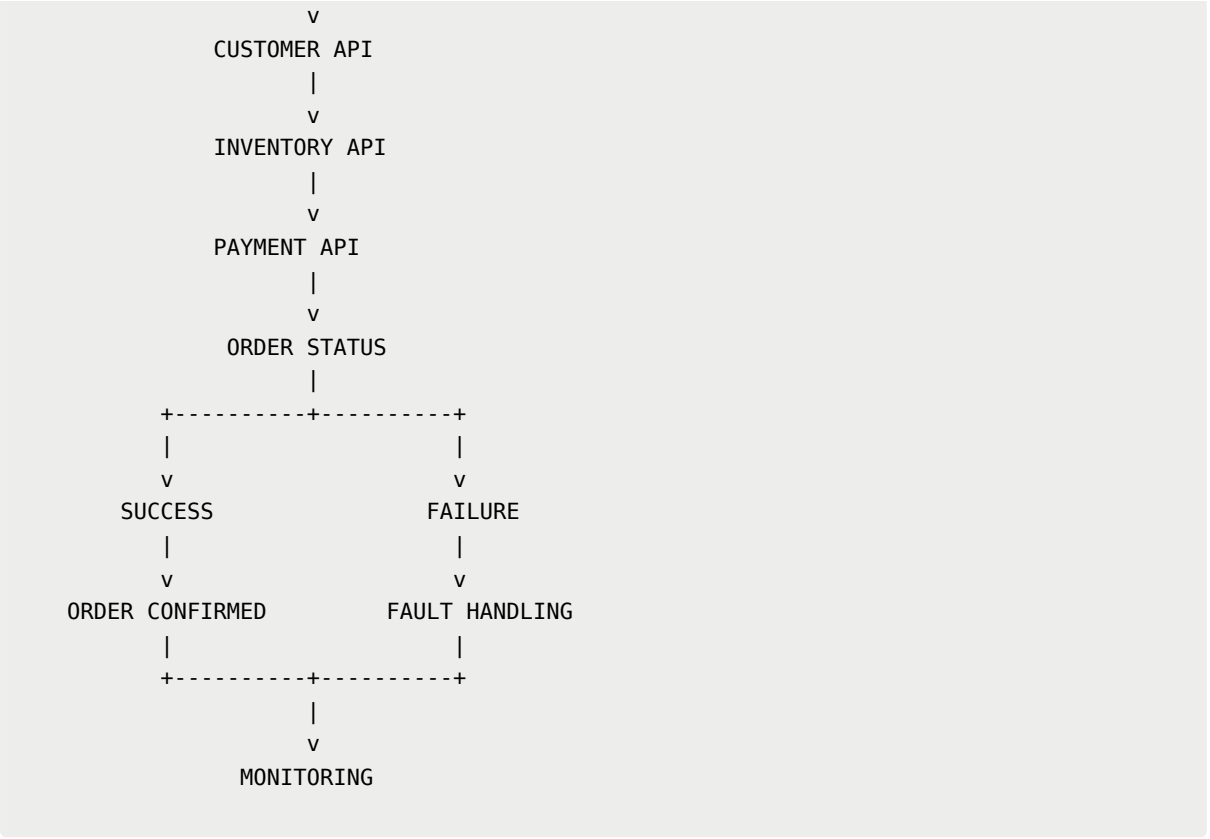
A retail organization wants to automate order processing.

When a customer submits an order:

1. Receive the order request.
2. Validate the order amount.
3. Verify customer status.
4. Check product availability.
5. Process payment.
6. Return order confirmation.
7. Handle failures.
8. Monitor execution.

Architecture

```
                CUSTOMER
                   |
                   v
            ORDER REQUEST
                   |
                   v
            CAI REST API
                   |
                   v
          PROCESS DESIGNER
                   |
                   v
          VALIDATE ORDER
                   |
```



Concepts Covered

CAI Topic	Usage in the Project
CAI Architecture	Coordinates the application interactions
API Management	Provides controlled API access and lifecycle management where configured
Process Designer	Implements order-processing workflow
Service Connectors	Connects customer, inventory, and payment services
Fault Handling	Handles invalid orders and service failures
Monitoring	Tracks process executions
Asset Management	Organizes processes and connectors

Expected Learning Outcome

After completing the case study, students should understand how CAI supports business process orchestration involving multiple application services.

PART 14: Interview Preparation – 25 Questions with Answers

The following questions are suitable for entry-level Informatica CAI interviews.

Section A: CAI Fundamentals

Q1. What is Informatica Cloud Application Integration?

CAI is a service within Informatica IDMC that enables organizations to integrate applications, automate business workflows, and communicate through APIs.

It is commonly used for application orchestration, service integration, and API-driven business processes.

Q2. What is the difference between CAI and CDI?

CDI primarily focuses on moving and transforming data between systems.

CAI primarily focuses on application communication, API integration, and business process orchestration.

Q3. What is process orchestration?

Process orchestration means coordinating multiple activities or services in a defined sequence to complete a business operation.

For example, order processing may involve customer verification, inventory checking, payment processing, and order confirmation.

Q4. What is an event-driven process?

An event-driven process starts in response to an event or request rather than relying only on a fixed schedule.

For example, receiving a new order request can initiate an order-processing workflow.

Q5. What is a synchronous process?

A synchronous process interaction is one in which the caller waits for a response.

For example, a customer verification API can receive a customer ID and return the verification result during the same request-response interaction.

Section B: Architecture

Q6. What are the major components of CAI?

The major components include Process Designer, Service Connectors, Connections, supported execution runtimes, and monitoring capabilities.

Together, these components support process development, connectivity, execution, and troubleshooting.

Q7. What is the role of the Secure Agent in CAI?

A Secure Agent can provide runtime connectivity and execution capabilities for supported application integration configurations, including access to systems within a private network.

The exact runtime behavior depends on the CAI deployment configuration.

Q8. What is the purpose of a connection?

A connection stores the configuration required to communicate with an external system.

Depending on the connector, this may include endpoint information, authentication details, and other connectivity properties.

Section C: API Management

Q9. What is an API?

An API is a defined interface that allows software applications to communicate and exchange information.

Q10. What is a REST API?

A REST API is an API designed according to REST architectural principles, commonly using HTTP methods to interact with resources.

Q11. What is the difference between GET and POST?

GET	POST
Primarily retrieves information	Submits data for processing or resource creation
Intended to be safe and read-only	May change application state

GET	POST
Commonly uses URL parameters	Commonly sends data in a request body
Example: Retrieve customer	Example: Create order

Q12. What is HTTP status code 401?

HTTP 401 indicates that the request lacks valid authentication credentials.

Q13. What is HTTP status code 404?

HTTP 404 indicates that the requested resource was not found.

Q14. What is API authentication?

API authentication verifies the identity of the application or user making a request.

Common mechanisms include API keys, Basic authentication, and OAuth-based authentication.

Section D: Process Designer

Q15. What is Process Designer?

Process Designer is the visual environment used to create application integration workflows.

It supports process steps, variables, business rules, service calls, and process outputs.

Q16. What is an Assignment step?

An Assignment step is used to assign or calculate values within a process.

For example, it can populate an approval status based on business logic.

Q17. What is a Decision step?

A Decision step evaluates conditions and directs process execution through the appropriate path.

For example, it can route a loan application to automatic approval or manual review.

Q18. What is a Service step?

A Service step invokes a configured service operation.

For example, it can call an external customer verification API.

Q19. What are Parallel Paths?

Parallel Paths allow independent branches of a process to execute concurrently, subject to the process design and runtime behavior.

For example, a process may send notifications to two independent services.

Section E: Service Connectors

Q20. What is a Service Connector?

A Service Connector defines the operations and message structures of an external service.

It enables CAI processes to invoke those operations through configured connections.

Q21. What is the difference between a Service Connector and a Connection?

A Service Connector defines the service interface and operations.

A Connection supplies the configuration needed to access the service.

Section F: Fault Handling and Monitoring

Q22. What is fault handling?

Fault handling is the process of detecting and managing errors during workflow execution.

It helps return meaningful error information and supports appropriate recovery actions.

Q23. How would you handle an API timeout?

I would first check the timeout details and determine whether the failure is temporary.

If retrying is safe and supported, I would apply the configured retry policy. Otherwise, I would return or record an appropriate failure and escalate the

issue according to operational procedures.

Q24. How do you troubleshoot a failed CAI process?

I would review the process instance in monitoring, identify the failed activity, inspect the fault information, and verify the relevant inputs and connection configuration.

I would document the root-cause findings and escalate issues that require permissions or production changes.

Q25. What is asset management in CAI?

Asset management is the practice of organizing, maintaining, and controlling integration assets such as processes, service connectors, and connections.

It supports reuse, maintainability, and controlled deployment.

Final Summary – To Remember

Topic	One-Line Understanding	Important Interview Keyword
CAI	Integrates applications and automates business workflows	Application Integration
CAI Architecture	Defines how design, connectivity, execution, and monitoring work together	Integration Architecture
API Management	Manages API access, security, usage, and lifecycle	API Governance
Process Designer	Creates business workflows visually	Process Orchestration
Service Connector	Defines external service operations	Service Integration
Connection	Configures access to external applications	Connectivity
Fault Handling	Manages errors during process execution	Exception Handling
Monitoring	Tracks execution and supports troubleshooting	Process Monitoring
Asset Management	Organizes and maintains integration components	Asset Lifecycle Management

Learning sequence:

CAI Fundamentals → CAI Architecture → REST APIs → Process Designer → Service Connectors → Fault Handling → Monitoring → Asset Management → End-to-End Business Project.

This sequence builds the required foundation before introducing more complex application integration scenarios.